과 제 계 획 서

Grad-Deploy

컨테이너 기반 인프라 자동 구축 솔루션

졸업 작품 / 소규모 프로젝트 인프라 설정의 장벽을 허물다

제출 일자	2026년
프로젝트명	Grad-Deploy (가칭)
구분	졸업 작품 과제 계획서 v2

1. 서비스 소개

1.1 서비스 개요

서비스명	Grad-Deploy (가칭) — 컨테이너 기반 인프라 자동 구축 솔루션
선택 배경	학생 개발자들이 졸업 작품 진행 시 DB, Redis, Backend, Frontend 등 다양한 기술 스택을 클라우드에 배포하는 과정에서 겪는 인프라 설정의 어려움을 해결하고자 함.
목표	복잡한 Kubernetes/Docker 설정 지식이 부족해도 프로젝트 구조만으로 최적화된 컨테이너 환경을 자동 구성하고 배포하는 것을 목표로 함.

1.2 핵심 가치

- ▶ 인프라 설정의 간소화 — 복잡한 YAML/CLI를 GUI로 대체
- ▶ 배포 생산성 향상 — 설정에 쏟던 시간을 서비스 로직 개발에 집중
- ▶ 표준화된 컨테이너 환경 제공 — 모범 사례(Best Practice) 기반 자동 생성
- ▶ 데모데이 완성도 보장 — 발표 현장에서 서비스가 살아있음을 실시간 증명 (신규)
- ▶ 크레딧 소진 방지 — 발표 전날 서비스 중단 최악의 상황 원천 차단 (신규)

1.3 핵심 기능

핵심 기능	상세 설명	기대 효과
멀티 컨테이너 자동 구성	Spring Boot, React, MySQL 등 기술 스택 선택 시 Docker Compose 및 K8s YAML 파일 자동 생성. 서비스 간 의존성(depends_on) 및 네트워크 자동 구성.	인프라 설정 시간 90% 단축, 설정 오류 원천 방지
원클릭 배포 파이프라인	GitHub 레포지토리 연동을 통해 소스 코드 변경 시 자동으로 컨테이너를 빌드하고 서버에 반영. CI/CD 파이프라인 자동 구성.	배포 자동화, 코드 변경 즉시 반영
리소스 모니터링 대시보드	현재 가동 중인 컨테이너의 CPU, 메모리 상태 시각화. 이상 징후 감지 시 알림 제공.	서버 자원 낭비 방지, 클라우드 비용 절감
환경 설정 템플릿	웹 커뮤니티, 데이터 분석, AI 모델 서버 등 졸업 작품 유형별 최적화된 템플릿 제공. Dockerfile ARG 기반 유연한 버전 관리.	빠른 프로젝트 시작, 검증된 설정 재사용
★ 데모데이 모드 (신규)	발표용 공개 URL을 생성하면 서비스 아키텍처 다이어그램, 컨테이너 상태, 업타임, API 응답속도를 한눈에 볼 수 있는 발표용 대시보드를 자동 생성. 심사위원이 URL만으로 서비스 완성도를 확인 가능.	발표 현장 신뢰도 향상, 배포 자체가 포트폴리오로 활용
★ 크레딧 번아웃 방지 (신규)	사용량 패턴을 분석해 크레딧 소진 예상 시점을 예측하고 발표일 D-day 알림 제공. 불필요한 컨테이너를 자동 슬립 모드로 전환하여 발표 전날 서비스 중단 방지.	발표 당일 서비스 가용성 보장, 클라우드 무료 크레딧 최대 활용

★ 표시 항목은 기존 서비스(Portainer, Railway, Render 등)에서 제공하지 않는 Grad-Deploy 신규 차별화 기능입니

다.

2. 타겟 사용자

2.1 주요 사용자

- ▶ 졸업 작품을 준비하는 IT 관련 학과 대학생 및 예비 개발자

2.2 사용자 페르소나

#	페르소나	상황 및 니즈
1	**인프라 초보 학생 개발자**	개발은 가능하지만 Docker나 클라우드 인프라 설정에 막막함을 느끼는 학생. "코드는 짰는데 배포를 어떻게 하지?"라는 막막함을 해소하고 싶어함.
2	**MSA 연동 팀 프로젝트 리더**	여러 개의 마이크로서비스(MSA)를 연동해야 하는데 네트워크 설정에서 어려움을 겪는 팀 프로젝트 리더. 서비스 간 의존성 설정 자동화가 필요함.
3	**클라우드 비용 절감 희망자**	한정된 클라우드 무료 크레딧 안에서 효율적으로 자원을 배분해 배포하고 싶은 사용자. 불필요한 리소스 낭비를 최소화하고 싶어함.
★ 4	졸업작품 발표 준비생 (신규)	발표 당일 서비스가 정상 작동함을 심사위원에게 증명해야 하는 학생. 발표 현장에서 아키텍처 다이어그램과 실시간 서비스 상태를 한눈에 보여주고 싶어함.

3. 기대 효과

인프라 구축 시간을 단축하여 사용자가 서비스 로직 개발에만 집중할 수 있는 환경을 조성함.

3.1 편의성 측면

- ▶ 파편화된 인프라 설정 문서를 찾는 대신, 통합 도구를 통해 신속하게 배포 환경 구축
- ▶ 오류 발생 시 즉각적인 로그 확인 및 원인별 해결 가이드 제공
- ▶ 포트 충돌 자동 감지 및 재할당으로 설정 실수 방지

3.2 학습적 측면

- ▶ 표준화된 설정 코드(ARG 기반 Dockerfile, depends_on 등)를 통해 올바른 Docker 활용법 학습
- ▶ 서비스 간 의존성 시각화로 컨테이너 네트워크 구조 직관적 이해

3.3 경제적 측면

- ▶ 효율적인 자원 관리로 클라우드 무료 크레딧 낭비 방지
- ▶ 리소스 모니터링 대시보드를 통한 과도한 컨테이너 리소스 사용 사전 차단

- ▶ ★ 크레딧 소진 예측 알림으로 발표일 전 서비스 중단 위험 사전 제거 (신규)

3.4 발표 및 포트폴리오 측면 (신규)

- ▶ ★ 데모데이 모드를 통해 심사위원이 URL 하나로 서비스 아키텍처와 실시간 운영 상태를 확인 가능
- ▶ ★ 배포 환경 자체가 포트폴리오로 활용되어 취업/대외 활동 시 차별화된 증거 자료로 활용
- ▶ ★ 발표 현장에서 서비스가 살아있음을 실시간으로 증명함으로써 심사 신뢰도 향상

4. 서비스 차별점

전문가 위주의 복잡한 클라우드 관리 서비스에서 탈피하여 학생 및 초보자 친화적인 인터페이스를 제공함.

4.1 기존 서비스와의 비교

구분	AWS / GCP	Terraform / Helm	Grad-Deploy (본 서비스)
타겟 사용자	엔터프라이즈 / 전문가	DevOps 엔지니어	학생 / 초보 개발자
러닝 커브	매우 높음	높음	낮음
초기 설정 시간	수 시간 ~ 수 일	수 시간	10분 이내
비용	유료 (복잡한 과금)	무료 (설정 복잡)	무료 / 경량
졸업작품 최적화	미제공	미제공	특화 템플릿 제공
데모데이 모드	미제공	미제공	★ 발표용 대시보드 자동 생성
크레딧 소진 방지	미제공	미제공	★ D-day 알림 + 자동 슬립 모드

4.2 본 서비스의 핵심 경쟁력

프로젝트 맞춤형 UX

복잡한 용어 대신 "어떤 DB를 쓰시나요?"와 같은 직관적인 문답 형식의 배포 마법사 제공. 선택 즉시 서비스 간 환경변수(예: SPRING_DATASOURCE_URL)가 자동으로 주입됨.

ARG 기반 유연한 버전 관리

Dockerfile에 ARG JAVA_VERSION, ARG APP_PORT를 사용해 빌드 시점에 버전과 포트를 오버라이드 가능. docker-compose의 build.args와 자동 연동.

GitHub 원클릭 Push

생성된 모든 파일(docker-compose.yml, Dockerfile, .env.example, .gitignore, README.md)을

GitHub 레포지토리에 직접 업로드. Classic PAT 토큰만으로 신속 연동.

★ 데모데이 모드 (신규 차별화)

발표용 공개 URL을 생성하면 서비스 아키텍처 다이어그램, 현재 컨테이너 상태, 업타임, API 응답속도를 한눈에 확인할 수 있는 발표용 대시보드를 자동으로 생성한다. 심사위원이 별도 설치 없이 URL만으로 서비스 완성도를 확인할 수 있으며, 배포 환경 자체가 포트폴리오 자료로 활용된다. Portainer, Railway, Render 등 기존 서비스에는 학생 발표를 위한 이러한 기능이 존재하지 않는다.

★ 크레딧 번아웃 방지 시스템 (신규 차별화)

사용량 패턴을 분석해 "이 속도라면 발표일 3일 전에 크레딧이 소진됩니다"와 같은 예측 알림을 제공한다. 발표일(D-day)을 설정하면 해당 날짜까지 서비스가 유지될 수 있도록 불필요한 컨테이너를 자동 슬립 모드로 전환하고, 크레딧 사용량 예측 그래프를 실시간으로 제공한다. 발표 전날 서비스가 내려가는 학생들의 가장 흔한 문제를 사전에 해결하는 기능으로, 기존 어떤 배포 플랫폼도 학생 발표 일정을 고려한 크레딧 관리 기능을 제공하지 않는다.

경량화된 자동화

대규모 엔터프라이즈 기능보다는 졸업 작품 규모에 최적화된 경량 컨테이너 오케스트레이션 로직 적용. 포트 충돌 자동 감지 및 의존성 그래프 시각화 제공.

5. 협업 방법 및 도구

분류	도구	용도
버전 관리	GitHub	Issue, Pull Request 활용한 코드 리뷰 및 버전 관리
일정 관리	Jira	스프린트 기반 태스크 관리 및 번다운 차트 활용
커뮤니케이션	KakaoTalk	긴급 공지 및 빠른 의사결정
음성 회의	Discord	화면 공유 및 음성 회의, 코드 리뷰

6. 향후 개발 계획

6.1 개발 로드맵

단계	기간	주요 작업	담당
1단계 기획	1 ~ 2주차	요구사항 명세서 작성, 배포 시나리오 설계, UI/UX 와이어프레임, 데모데이 모드 화면 설계, 크레딧 예측 알고리즘 기획	전원
2단계 개발	3 ~ 8주차	프론트엔드 대시보드 개발, 백엔드 자동 설정 엔진 구현, Docker SDK 연동, 크레딧 사용량 추적 모듈 개발	역할 분담
3단계 연동	9 ~ 12주차	GitHub API 연동, 클라우드 API 테스트, 모니터링 대시보드 구현, 데모데이 공개 URL 생성 기능 구현	전원
4단계 검증	13 ~ 16주차	통합 테스트, 졸업작품 유형별 시나리오 검증, 데모데이 UX 테스트, 크레딧 예측 정확도 검증, 성능 최적화	전원

6.2 단계별 세부 계획

- ▶ 요구사항 명세서 작성 및 배포 시나리오 설계
- ▶ 프론트엔드(대시보드) 및 백엔드(자동 설정 엔진) 역할 분담
- ▶ Docker SDK 및 클라우드 API 연동 테스트 프로세스 수립
- ▶ 데모데이 모드: 발표용 공개 대시보드 URL 생성 기능 설계 및 구현 (신규)
- ▶ 크레딧 번아웃 방지: 사용량 패턴 분석 및 D-day 기반 자동 슬립 모드 구현 (신규)

6.3 기술 스택 (예정)

계층	기술	선정 이유
Frontend	React + TypeScript	컴포넌트 기반 UI, 타입 안전성 보장
Backend	Spring Boot / Node.js	REST API 서버, GitHub API 연동
Container	Docker + Docker Compose	멀티 컨테이너 오케스트레이션
Orchestration	Kubernetes (K8s)	프로덕션 레벨 컨테이너 관리
Monitoring	Prometheus + Grafana	컨테이너 리소스 모니터링, 크레딧 사용량 추적
CI/CD	GitHub Actions	자동 빌드/배포 파이프라인

Grad-Deploy — 인프라 설정의 장벽을 허물고, 개발에만 집중하게 합니다.